Système Multi-Agents d'IA Générative pour l'Automatisation de la Livraison Logicielle

Challenge proposé par

Équipe 01

Oumeyma RHENNOU
Elhadji Mouhamadou Mactar GUEYE
Akram ZAHRY
Manal BOUKRIM

Dépôt GitHub

1 Introduction

La livraison logicielle constitue aujourd'hui l'un des goulots d'étranglement les plus coûteux des organisations numériques. Un cycle de développement standard — de la spécification à la mise en production — mobilise en moyenne entre six et dix-huit mois de travail humain spécialisé, réparti entre architectes, développeurs backend, développeurs frontend, ingénieurs DevOps et équipes QA.

Selon une étude Gartner de 2025, plus de 70% du temps de ces équipes est consacré à des tâches répétitives et structurées : rédaction de spécifications techniques, mise en place d'environnements de développement, génération de code boilerplate, configuration des pipelines CI/CD, et écriture de tests unitaires.

Ces tâches, bien qu'indispensables, ne créent pas de valeur différenciante pour les organisations. Elles mobilisent des compétences rares et coûteuses pour produire des artefacts largement prévisibles, dont la structure varie peu d'un projet à l'autre. C'est précisément ce constat qui a motivé la conception du pipeline SDLC multi-agents.

Notre solution repose sur un pipeline d'agents IA orchestrés par LangGraph qui transforme un besoin exprimé en langage naturel en une application complète — architecture, code backend, interface frontend, CI/CD et tests — en quelques minutes, sans intervention humaine.

1.1 Innovation et originalité

La plupart des systèmes multi-agents (comme AutoGen ou CrewAI) reposent sur une communication par messages entre agents. Notre approche propose un modèle différent.

Le système est composé de cinq agents spécialisés qui s'exécutent en séquence, chacun lisant et enrichissant un état partagé — l'AgentState — sans jamais s'appeler directement entre eux, assurant la cohérence et la traçabilité.

Un agent unique (Architect) génère un blueprint central (JSON) servant de source de vérité partagée. Le système permet également la génération automatique de diagrammes C4, des logs, de la documentation et du tech stack, offrant une automatisation complète, cohérente et observable.

2 Qualité Technique et Faisabilité

2.1 Robustesse technique de la solution

Notre architecture applique une séparation stricte des responsabilités (état, logique métier, routage, orchestration), ce qui limite les régressions. Les agents interagissent via un état partagé typé, sans couplage direct : le système reste ainsi lisible, testable et maintenable, même à grande échelle.

Scalabilité native par conception. Chaque sous-graphe est un nœud autonome : ajouter un

nouveau domaine revient à connecter un nœud, sans toucher au reste.

Gestion des cycles sans compromis. Les boucles de retry sont des arêtes conditionnelles natives, conçues pour les flux itératifs qu'un système de génération autonome requiert inévitablement.

État persistant et auditable. L'état complet du système est accessible, typé et traçable à chaque étape : aucun effet de bord, une reproductibilité totale.

Contrôle des agents par fichiers de configuration JSON Les agents sont pilotés par des fichiers JSON structurés définissant rôles, contraintes et format de sortie : modifiables sans toucher au code. Changer de stack ou de modèle LLM revient à ajuster une ligne de configuration, pas à refactoriser le pipeline.

Justification des choix techniques : LangGraph pour l'orchestration, modèles LLM interchangeables, architecture en couches découplées : chaque choix répond à une contrainte précise, pas à une préférence.

Pistes innovantes : Un architect agent transforme une description en langage naturel en blueprint JSON expert, effaçant la frontière entre intention métier et implémentation technique.

Architecture Mosaïque : Chaque sous-graphe est indépendant et interchangeable : ajouter un agent DevOps ou retirer le frontend se résume à connecter deux arêtes, sans aucune refactorisation.

Qualité et lisibilité : Responsabilité unique par composant, typage systématique, zéro couplage entre agents. Le système se lit comme sa propre documentation.

Intégration aux systèmes existants : Interface universelle : JSON en entrée, projet structuré en sortie. Compatible nativement avec tout workflow existant : CLI, API REST, CI/CD : sans modifier le pipeline.

Réalisme à grande échelle : Chaque sous-graphe étant isolé et stateless, il peut être déployé sur des machines distinctes et exécuté en parallèle. La performance scale proportionnellement aux ressources allouées.

3 Impact, risques et responsabilité

3.1 Risques et mitigation

Confidentialité et fuite de données

Le système multi-agents manipule des informations de complexité croissante (blueprint, base de données, code, infrastructure), augmentant le risque d'exposition vers des APIs tierces et compromettant la propriété intellectuelle.

Mesures :

- Hébergement local des modèles (Ollama, vLLM)
- Limitation stricte du contexte transmis
- Adaptation des agents aux standards internes

Erreurs en cascade

Une erreur initiale se propage à l'ensemble du pipeline, générant des incohérences difficiles à détecter. Chaque agent intègre une auto-correction, mais des validations externes restent nécessaires.

Mesures :

- Validation humaine aux étapes critiques (*human-in-the-loop*)

3.2 Impact environnemental

L'exécution séquentielle de plusieurs agents et appels à des modèles lourds entraîne une empreinte carbone élevée, problématique à grande échelle.

Mesures :

- Utilisation de modèles légers pour les tâches simples
- Suivi de l'empreinte (ex. CodeCarbon)

3.3 Limites et supervision

Les agents optimisent localement sans garantir la cohérence globale. La qualité dépend du blueprint initial et les incohérences métier ne sont pas détectées automatiquement.

De plus, l'utilisation de modèles gratuits et de quotas limités en tokens a restreint les performances du système (qualité des générations, profondeur du raisonnement, stabilité). Dans un contexte industriel, l'usage de modèles plus performants, spécialisés ou fine-tunés permettrait d'améliorer significativement la fiabilité et la qualité des résultats.

L'outil assiste le développement sans le remplacer : une validation humaine reste indispensable avant mise en production.

4 Conclusion et recommandations

Ce projet propose une architecture multi-agents modulaire permettant de transformer une intention métier en application logicielle complète. L'approche repose sur une séparation stricte des responsabilités et une orchestration par graphe, garantissant lisibilité, maintenabilité et extensibilité.

Apports principaux.

L'architecture développée présente plusieurs atouts majeurs. D'une part, la scalabilité est assurée par une conception modulaire où chaque sous-graphe constitue un composant autonome. D'autre part, l'utilisation d'un état partagé typé permet une traçabilité complète et une reproductibilité des traitements. Le pilotage des agents via des fichiers de configuration JSON offre une grande flexibilité, facilitant l'adaptation à différents modèles ou stacks techniques sans refonte du système. Enfin, l'intégration native avec des workflows existants (API, CLI, CI/CD) renforce la faisabilité d'un déploiement en environnement réel.

Limites identifiées.

Malgré ces avantages, certaines limites subsistent. La dépendance aux modèles de langage introduit des risques d'erreurs en cascade et d'incohérences globales. De plus, l'absence de compréhension métier par les agents limite leur capacité à détecter des erreurs conceptuelles. Enfin, le coût computationnel et l'empreinte environnementale peuvent devenir significatifs dans un contexte d'utilisation intensive.

Recommandations.

Pour améliorer la solution, plusieurs axes sont proposés :

- Intégrer des mécanismes de validation intermédiaires, incluant une supervision humaine aux étapes critiques
- Réduire la dépendance aux modèles lourds en privilégiant des modèles spécialisés ou plus légers
- Renforcer les tests automatisés et la validation des sorties de chaque agent
- Sécuriser davantage les échanges de données, notamment via un hébergement local des modèles

Perspectives d'évolution.

À court terme, l'amélioration des mécanismes de contrôle et de validation permettra d'augmenter la robustesse globale du système. À moyen terme, l'intégration d'agents spécialisés (DevOps, sécurité, optimisation) et l'exécution parallèle des sous-graphes ouvriront la voie à une automatisation plus complète et performante. Enfin, l'évolution vers des modèles plus fiables et mieux adaptés au contexte métier pourrait réduire significativement les limites actuelles.

En conclusion, cette approche constitue une avancée prometteuse vers l'automatisation du développement logiciel. Elle ne remplace pas l'expertise humaine, mais agit comme un accélérateur puissant, à condition d'être encadrée par des mécanismes de contrôle adaptés.